

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI – 590018**



An Internship Report on
“Namma-Santhe Ledger – Digital Khata App for Rural Vendors”

Submitted in partial fulfilment of the requirement for the award
Degree of Bachelor of Engineering
in
Computer Science & Engineering

Submitted by
SM VINAY KUMAR
(1AT22CG096)

Internship carried out at



Internal Guide
Prof. Poornima T
Assistant Professor



2025-26

ATRIA INSTITUTE OF TECHNOLOGY
Department of Computer Science & Engineering
Anandanagar, Bangalore-560024



ATRIA INSTITUTE OF TECHNOLOGY
Anandanagar, Bangalore-560024

CERTIFICATE

This is to certified that this internship work presented by **SM VINAY KUMAR** bearing **USN - 1AT22CG096** of **Atria Institute of Technology, Bangalore** in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi during 2025-26. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the department library. The internship report has been approved as it satisfies the academic requirements with respect to the internship report as prescribed for the said Degree.

Signature of Guide
Prof. Poornima T

Signature of HOD
Dr. Devi Kannan

Signature of Principal
Dr. Rajesha S

Name of Examiners with Signature & Date

1.

2.

Declaration

I, SM VINAY KUMAR (1AT22CG096) hereby declare that this internship work is carried out under the guidance of **Prof. Poornima T** Assistant Professor, Dept. of CS&E. This internship work is submitted to Visvesvaraya Technological University in partial fulfilment of the requirement for the award of degree of Bachelor of Engineering in Computer Science & Engineering for the academic year 2025-26.

Place: Bangalore

Date:

Signature of Student

Acknowledgement

I would like to express my sincere gratitude to the respected Principal **Dr. Rajesha S.** , for providing a congenial environment to work in. I also like to express my sincere gratitude to **Dr. Devi Kannan, Head of Department, Computer Science & Engineering**, for her continuous support and encouragement. I am indeed indebted to **Prof. Poornima T**, coordinator and guide for her continued support, advice and valuable inputs during the course of this internship work. Last, but not the least I would like to thank my family, who have acted as a beacon of light throughout my life. My sincere gratitude goes out to all my comrades and well-wishers who have supported me through all the ventures.

Abstract of Internship

This internship focuses on the development of an Android application integrated with modern concepts such as Generative Artificial Intelligence. It was conducted through a structured, project-based learning approach with the primary objective of gaining hands-on experience in building real-world mobile applications. During the initial phase, fundamental Android concepts such as user interface design, activity lifecycle management, navigation between screens, and efficient data handling were covered. These concepts were reinforced through mini-projects, helping to build a strong foundation in Kotlin-based Android development.

The major project undertaken during this internship is titled “Namma-Santhe Ledger – Digital Khata App for Rural Vendors.” The application aims to provide a simple and reliable digital solution for small vendors in weekly markets (Santhe), where transactions are often recorded using informal credit systems (udari). Traditional methods like handwritten notes or memory can lead to errors and financial losses. This application addresses the problem by enabling vendors to digitally record transactions, manage customer information, and track outstanding balances in an organized manner.

The application includes key features such as quick transaction entry, customer management, and a clear ledger view for tracking credit and payments. It also supports invoice generation, basic reporting, and WhatsApp-based reminders to improve payment collection. A QR-based transaction confirmation mechanism is included to enhance transparency and reduce disputes, while Firebase Authentication ensures secure user access. An OCR-based feature is implemented to scan handwritten bills and convert them into digital entries. However, due to challenges in recognizing multilingual handwritten text (especially Kannada and Hindi), the application provides a manual correction interface to verify and edit extracted data before saving.

The system is designed with an offline-first approach, allowing core functionalities such as transaction entry, invoice generation, reporting, and OCR processing to work without internet connectivity, making it suitable for rural environments. The application is developed using Kotlin and follows a modular architecture to ensure scalability and maintainability, with a user interface based on Material Design principles for simplicity and ease of use. Future enhancements include improving OCR accuracy, strengthening QR-based verification, and adding cloud synchronization for backup and multi-device access. Overall, this internship provided valuable exposure to real-world development, problem-solving, and modern workflows, bridging the gap between academic learning and practical implementation.

Table of Contents

Chapter 1- Introduction.....	7
1.1 Overview of Internship	
1.2 Objectives of Internship	
1.3 Scope of Work	
1.4 Methodology / Approach	
Chapter 2- Organization Profile	11
2.1 Introduction to Organization	
2.2 Vision and Mission	
2.3 Products / Services	
2.4 Organizational Structure	
Chapter 3- Internship Work Description.....	14
3.1 Project Title / Domain	
3.2 Tools and Technologies Used	
3.3 System Requirements (if applicable)	
3.4 Detailed Work Description	
3.5 Implementation / Process Flow	
Chapter 4-Results and Analysis	19
4.1 Outcomes of Internship	
4.2 Performance Analysis	
4.3 Challenges Faced	
4.4 Solutions Implemented	
Chapter 5: Learning Outcomes	22
5.1 Technical Skills Gained	
5.2 Soft Skills Developed	
5.3 Practical Exposure	
Chapter 6: Conclusion and Future Scope	25
6.1 Conclusion	
6.2 Future Enhancements / Scope	

Appendices

- A: Screenshots / Code
- B: Daily Log / Work Diary signed by industry mentor
- C: Certificates

Chapter 1: Introduction

1.1 Overview of Internship

The internship undertaken during the academic year 2025–26 was focused on developing practical skills in Android application development along with exposure to modern technologies such as Generative Artificial Intelligence (GenAI). The internship was conducted through a structured learning platform where students were introduced to real-world development practices through guided modules, mini-projects, and a final major project.

In the initial phase of the internship, various foundational concepts of Android development were covered. These included understanding the architecture of Android applications, designing user interfaces using XML, handling user interactions, implementing navigation between screens, and managing data within the application. The learning approach was primarily hands-on, ensuring that each concept was reinforced through practical implementation.

The internship also emphasized the importance of modern development workflows. Students were introduced to tools such as Android Studio for application development, Visual Studio Code for assisted coding using AI tools, and version control concepts. Additionally, AI-based tools were used to enhance productivity, particularly in code generation, debugging, and structuring applications.

As part of the learning process, multiple mini-projects were completed. These mini-projects played a crucial role in building a strong foundation, as they helped in understanding individual components such as RecyclerView, adapters, layouts, and activity navigation. Each mini-project focused on a specific functionality, which later contributed to the development of the final major project.

The major project developed during the internship is titled “Namma-Santhe Ledger – Digital Khata App for Rural Vendors.” The purpose of this application is to provide a digital solution for small vendors in weekly markets (Santhe), where transactions are often managed through informal credit systems (udari). The application enables vendors to record transactions, manage customer data, and track outstanding balances in an organized manner. It includes features such as quick transaction entry, customer ledger management, invoice generation, WhatsApp-based reminders, QR-based transaction confirmation, and OCR-based bill scanning to digitize handwritten records, with manual correction to ensure accuracy.

Overall, the internship provided a comprehensive learning experience by combining theoretical understanding with practical application. It enabled the development of both technical and problem-solving skills, preparing students for real-world software development scenarios.

1.2 Objectives of Internship

The primary objective of this internship is to provide hands-on experience in developing Android applications while incorporating modern technologies such as Generative Artificial Intelligence. The internship focuses on building practical skills by working on real-world problems and developing a complete mobile application.

The internship aims to bridge the gap between academic learning and industry requirements by emphasizing practical implementation, user-centric design, and real-world problem solving. A

key objective is to develop a strong understanding of Android application development using the Kotlin programming language, including designing intuitive user interfaces, handling user interactions, managing application components, and ensuring smooth navigation across multiple screens.

Another important objective is to understand the complete software development lifecycle, including requirement analysis, system design, implementation, testing, and deployment. Through the development of the “Namma-Santhe Ledger” application, students gain experience in building an offline-first system, managing local data efficiently, and implementing features such as transaction recording, invoice generation, and customer management.

The internship also aims to introduce the integration of modern technologies such as Generative AI and OCR-based data processing. This includes understanding how AI and automation can enhance user experience by simplifying tasks such as digitizing handwritten bills and generating structured data, while also addressing real-world challenges such as accuracy and multilingual input handling.

Additionally, the internship focuses on improving problem-solving and debugging skills. Students are encouraged to identify issues in application logic, optimize performance, and implement reliable solutions. This helps in developing strong analytical thinking and technical confidence.

Another objective is to enhance soft skills such as time management, independent learning, adaptability, and structured development practices. Since the project is developed in multiple phases, it requires proper planning, execution, and iterative improvement.

In summary, the internship aims to:

- Build strong technical skills in Android application development using Kotlin
- Provide hands-on experience in developing a real-world application
- Introduce modern technologies such as Generative AI and OCR integration
- Enhance problem-solving, debugging, and analytical skills
- Develop understanding of offline-first application design
- Prepare students for industry-level software development practices

1.3 Scope of Work

The scope of the internship project includes the design and development of an Android application that focuses on simplifying financial record management for small vendors in weekly markets (Santhe). The application provides a user-friendly and efficient platform for recording transactions, managing customer data, and tracking credit (udari) in a structured digital format.

The core functionality of the application includes a customer and transaction management system, where vendors can add customers, record credit and payment transactions, and view a clear ledger of all activities. Each transaction is stored and organized in a way that allows users to easily track outstanding balances and monitor daily business performance.

Another important component of the application is the invoice and reporting module. This module enables users to generate simple invoices and view basic reports related to daily transactions, pending amounts, and payment history. Additionally, WhatsApp-based reminders can be used to notify customers about pending dues, improving payment collection efficiency.

The application also includes an OCR-based feature that allows users to scan handwritten bills and convert them into digital entries. Since handwritten multilingual text (Kannada, Hindi, and English) may not always be accurately recognized, the system provides a manual correction interface to ensure data accuracy before saving. A QR-based transaction confirmation feature is also included to improve transparency and reduce disputes between vendors and customers.

The scope of the project also includes designing a clean and intuitive user interface using XML-based layouts and Material Design principles. The application is developed with an offline-first approach, ensuring that core functionalities such as transaction entry, invoice generation, reporting, and OCR processing work without internet connectivity.

While the initial phase focuses on core functionality, the future scope includes enhancing OCR accuracy, integrating Generative AI for intelligent insights, and enabling optional cloud synchronization using Firebase for backup and multi-device access. The project is designed to be scalable, allowing additional features to be incorporated in future development phases.

1.4 Methodology / Approach

The development of the project followed a structured and systematic approach to ensure efficient implementation and reliable performance. The methodology was divided into multiple phases, each focusing on a specific aspect of the application development lifecycle.

The first phase involved requirement analysis and problem understanding. In this phase, the core objective of developing a digital ledger application for small vendors was clearly defined. Key features such as transaction management, customer tracking, invoice generation, OCR-based bill scanning, and QR-based confirmation were identified. Special consideration was given to designing the application as an offline-first system suitable for rural environments.

The second phase focused on system design and planning. The overall architecture of the application was designed using an MVVM-based approach to ensure modularity and maintainability. User flows were defined for key operations such as adding transactions, viewing ledgers, scanning bills, and generating invoices. The user interface was planned using XML layouts following Material Design principles to ensure simplicity and ease of use.

The third phase involved implementation and development. The application was developed using Kotlin in Android Studio. Core modules such as customer management, transaction handling, and ledger display were implemented using local storage. OCR functionality was integrated to extract data from handwritten bills, followed by parsing and manual correction to ensure accuracy. A QR-based confirmation feature was implemented to improve transparency in transactions.

In addition, cloud-related components were incorporated using Google Cloud Platform (GCP) and Firebase services. Firebase Authentication was used for secure user login, while Google Cloud IAM (Identity and Access Management) concepts were applied to manage roles and permissions for accessing cloud resources. These services were designed as optional layers to support secure access, data backup, and future scalability without affecting the core offline functionality of the application.

The fourth phase focused on testing and debugging. The application was tested across different scenarios to ensure smooth functionality, especially in offline conditions. Issues related to data handling, OCR inaccuracies, UI responsiveness, and navigation were identified and resolved.

Edge cases such as incorrect OCR results and invalid inputs were handled through validation and manual correction interfaces.

The final phase involved evaluation and future enhancement planning. Based on testing results, potential improvements were identified, including enhancing OCR accuracy for multilingual handwritten inputs, strengthening QR-based validation mechanisms, and expanding cloud integration for backup and multi-device synchronization using GCP services.

Overall, the methodology ensured a structured and organized development process, improving reliability, maintainability, and usability of the application while addressing real-world constraints.

Chapter 2: Organization Profile

2.1 Introduction to Organization

The internship was carried out through *Mindmatrix*, a technology-focused training and development platform located in Bengaluru. The organization is known for providing structured, industry-oriented training programs that help students transition from academic learning to real-world software development.

Mindmatrix operates with the objective of creating an environment that closely simulates professional software development practices. Unlike traditional classroom learning, the platform emphasizes practical implementation, project-based learning, and continuous skill development. Interns are encouraged to actively participate in building applications, solving real-world problems, and understanding the complete software development lifecycle.

The training environment is designed to expose students to modern technologies such as Android development, cloud computing, and Generative Artificial Intelligence. The focus is not only on learning tools and programming languages but also on understanding how to apply them effectively in real applications.

The internship provided an opportunity to work on a major project, which involved designing and developing an Android application from scratch. This helped in gaining hands-on experience and understanding how theoretical concepts are applied in real-world scenarios.

2.2 Vision and Mission

Vision

The vision of Mindmatrix is to empower students with industry-relevant technical skills and prepare them to meet the demands of the rapidly evolving technology sector. The organization aims to bridge the gap between academic knowledge and practical implementation by providing real-world exposure to students.

Mission

The mission of the organization is to deliver high-quality training through hands-on learning and project-based approaches. It focuses on:

- Providing practical exposure to modern technologies
- Encouraging problem-solving and analytical thinking
- Helping students build complete applications
- Preparing students for professional software development environments

The organization strives to ensure that students are not only able to understand concepts but also apply them effectively in real-world situations.

2.3 Products and Services

Mindmatrix offers a variety of training programs and services designed to enhance technical skills and provide industry exposure.

Internship Programs

The organization provides structured internship programs that focus on real-world application

development. These programs include guided learning, project implementation, and continuous evaluation through reviews.

Project-Based Learning

Students are required to work on mini-projects and a major project during the internship. This helps in understanding individual components as well as complete system development.

Technology Training

The organization provides training in various domains such as:

- Android application development
- Web development
- Cloud computing
- Generative Artificial Intelligence

AI-Based Learning Tools

Students are encouraged to use AI tools for coding assistance, debugging, and improving productivity. This introduces them to modern development workflows used in the industry.

2.4 Organizational Structure

The organizational structure of Mindmatrix is designed to support efficient learning, structured guidance, and successful project execution through a well-defined hierarchy and collaborative environment.

Mentors and Trainers

Mentors play a crucial role in guiding students throughout the internship. They provide technical support, explain complex concepts, and assist in resolving development challenges. Trainers conduct structured sessions to introduce core topics such as Android development, application architecture, and modern tools. They also ensure that students follow best practices in coding and design.

Project Guides

Each student is assigned a project guide who continuously monitors the progress of the project. The guide provides regular feedback, evaluates performance, and ensures that the project aligns with defined objectives and quality standards. They also assist in refining features, improving implementation, and preparing students for project reviews and evaluations.

Students / Interns

Students actively participate in all learning and development activities. They are responsible for understanding concepts, completing assigned tasks, and developing the application independently. Interns are expected to demonstrate progress through regular updates, implement feedback, and maintain consistency in their work throughout the internship.

Review and Evaluation Process

The organization follows a periodic review system where students present their progress, demonstrate features, and receive constructive feedback. This helps in identifying gaps, improving performance, and ensuring timely completion of the project. Continuous evaluation also encourages accountability and structured development.

Collaborative Learning Environment

Mindmatrix promotes a collaborative learning culture where students can interact, share ideas, and solve problems collectively. Peer discussions, knowledge sharing, and group problem-solving sessions enhance understanding and encourage teamwork.

Tools and Support System

The organization provides access to essential tools such as Android Studio for development and AI-assisted tools for coding support and debugging. Students are encouraged to use these tools effectively to improve productivity and code quality.

Workflow and Development Approach

The internship follows a phased workflow, including learning, implementation, testing, and review. This structured approach ensures that students gradually build their skills while working on real-world applications, maintaining both technical depth and practical relevance.

Overall, the organizational structure of Mindmatrix ensures a balanced combination of guidance, independence, and collaboration, enabling students to effectively develop technical skills and complete their projects successfully.

Chapter 3: Internship Work Description

3.1 Project Title and Domain

The project undertaken during the internship is titled “*Namma-Santhe Ledger – Digital Khata App for Rural Vendors*.” The primary aim of this project is to develop a mobile application that simplifies financial record management for small vendors in weekly markets (Santhe). In many such environments, transactions are maintained through informal credit systems (udari), where vendors rely on memory or handwritten notes. This often leads to errors, missed payments, and financial losses. The application addresses this problem by providing a structured, digital, and user-friendly platform for recording transactions, managing customers, and tracking outstanding balances.

The domain of the project lies in Android application development, with an extended scope toward intelligent data processing and automation. The application is developed using Kotlin and XML-based layouts, focusing on building a robust and scalable mobile solution. It incorporates features such as transaction management, invoice generation, OCR-based bill scanning, and QR-based transaction confirmation. These features aim to improve accuracy, transparency, and efficiency in managing day-to-day financial activities.

The project is designed not only to demonstrate technical implementation but also to solve a real-world problem in the informal retail sector. By adopting an offline-first approach and a simple, intuitive interface, the application ensures usability even in low-connectivity environments. Future enhancements include improving OCR accuracy for multilingual handwritten inputs and integrating advanced AI-based features for better insights and automation, thereby increasing the practical value of the application.

3.2 Tools and Technologies Used

The development of the application involved the use of several modern tools and technologies, each playing a crucial role in different stages of the development process. The primary development environment used for this project is Android Studio, which provides a comprehensive platform for designing, coding, debugging, and testing Android applications. It offers features such as layout editors, real-time previews, and advanced debugging tools, which significantly improve development efficiency.

Kotlin was used as the primary programming language for implementing the application logic. Kotlin is officially supported for Android development and provides advantages such as concise syntax, null safety, and improved readability. These features help reduce common programming errors and make the code more maintainable and scalable.

The user interface of the application was designed using XML layouts along with Material Design components. XML enables structured and flexible UI development, ensuring proper alignment and responsiveness across different screen sizes. Material Design components such as CardView, Toolbar, and Buttons were used to create a clean, intuitive, and user-friendly interface suitable for all users, including those with minimal technical experience. For data management, Room Database was used to implement an offline-first storage system. It allows efficient handling of structured data such as customers, transactions, and invoices, ensuring that the application works seamlessly even without internet connectivity. Gradle was used as the build automation tool for dependency management and project configuration, ensuring smooth compilation and consistent builds.

Additional technologies were integrated to enhance functionality. Google ML Kit was used for OCR (Optical Character Recognition) to extract text from handwritten bills, which is then processed and converted into structured data. QR code generation and scanning libraries were used to implement transaction confirmation features, improving transparency. Firebase Authentication was integrated to provide secure user login and basic access control. AI-assisted development tools were also used during the project to support code generation, debugging, and structuring of the application. These tools helped improve productivity and allowed faster implementation of features while maintaining code quality. Overall, the combination of these tools and technologies ensured the development of a robust, scalable, and user-friendly application capable of addressing real-world problems effectively.

3.2 System Requirements

The development and execution of the application require both hardware and software resources. These requirements are divided into two categories: development environment requirements and application runtime requirements.

Development Hardware Requirements:

A system with a minimum of 8 GB RAM was used to ensure smooth operation of Android Studio and related development tools. Since Android development environments are resource-intensive, sufficient memory and processing capability are essential for efficient performance.

Testing Device Requirements:

An Android smartphone was used for testing the application. Testing on a real device provides a more accurate representation of application behavior compared to emulators. The device was configured with developer options enabled, including USB debugging, to allow direct installation and testing of the application.

Software Requirements:

The primary development tool used is Android Studio, which provides an integrated environment for coding, debugging, and testing. Kotlin is used as the programming language for application development. Gradle is used as the build automation tool for managing dependencies and compiling the project. The development environment is set up on a Windows operating system with a stable internet connection for downloading dependencies and updates.

Application Runtime Requirements:

The application is designed to run on Android smartphones with a minimum of 3 GB RAM. It requires camera support for QR scanning and sufficient storage for maintaining the local database and generated files such as invoices. The application is optimized to function efficiently on low-end devices.

External Dependencies:

The system integrates Firebase Authentication for secure user login. ZXing library is used for QR code generation and scanning, and iText7 is used for generating PDF invoices.

System Constraints:

The application follows an offline-first approach, meaning core functionalities such as transaction entry, ledger viewing, and invoice generation must work without internet connectivity. The user interface is designed to be simple and intuitive to support users with minimal technical knowledge.

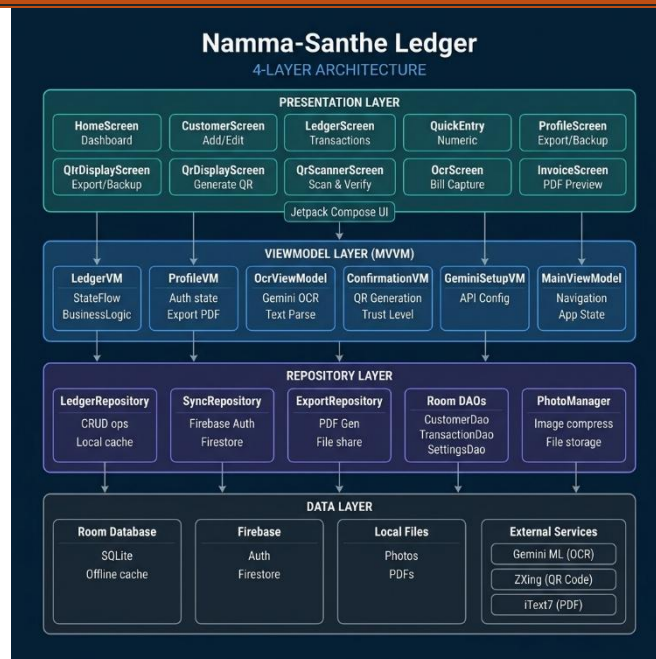


Figure : System Architecture Diagram
(Insert Architecture Image Here - 4-Layer Architecture)

3.3 Detailed Work Description

The development of the Namma-Santhe Ledger application was carried out using a modular and layered architecture. The system consists of multiple interconnected modules such as customer management, transaction handling, QR-based confirmation, invoice generation, and reporting.

The application begins with a dashboard that provides an overview of total outstanding balances and quick access to major functionalities. The customer module allows vendors to add, edit, and search customers, including storing profile photos for easy identification.

The core functionality lies in the ledger module, where transactions such as credit (udari) and payments are recorded. Each transaction is stored locally using Room Database, ensuring offline availability and fast access.

A QR-based confirmation mechanism is implemented to enhance transparency. After a transaction is recorded, a QR code is generated containing transaction details. This QR can be scanned for verification, reducing disputes and improving trust between vendor and customer.

Additionally, the application includes invoice generation and export features. Vendors can generate structured PDF invoices and share them via external platforms such as WhatsApp.

The system also supports optional OCR-based input for capturing handwritten bills. However, due to limitations in multilingual handwriting recognition, manual correction is provided to ensure data accuracy.

USE CASE DIAGRAM

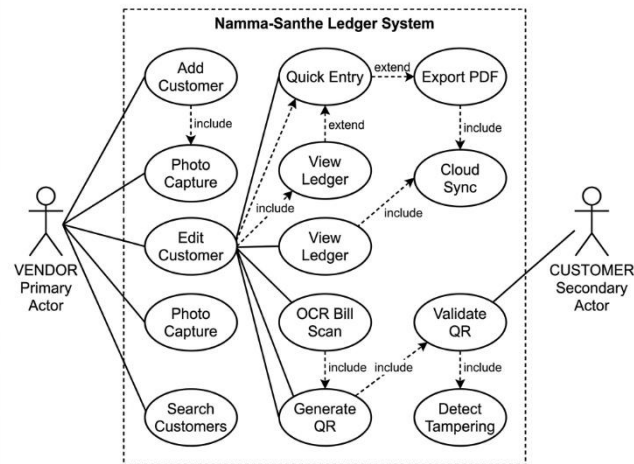


Figure : Use Case Diagram

UML CLASS DIAGRAM

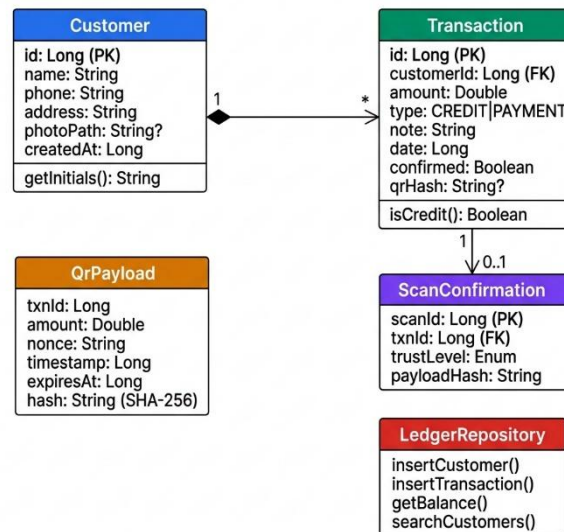


Figure : UML Class Diagram

3.4 Implementation and Process Flow

The application follows a structured workflow designed for quick and efficient transaction handling. The process begins at the home screen, where the vendor can view a summary of key information such as total outstanding balance, daily transactions, and overdue amounts. From this screen, the user can navigate to the customer list, select an existing customer, or add a new one. Once a customer is selected, the application moves to the ledger screen, where all transaction details related to that customer are displayed. The ledger screen presents a clear view of both credit (udari) and payment transactions. The vendor can add a new transaction using the quick entry interface by entering the amount and selecting the transaction type. After saving,

the system immediately updates the customer's balance and stores the transaction in the local database. This ensures that the data is always consistent and available even without internet connectivity.

Following the transaction entry, a QR code is generated automatically containing key transaction details such as amount, timestamp, and transaction ID. This QR code can be scanned for validation, providing an additional layer of transparency between vendor and customer. In addition, the application supports invoice generation, where transaction data is formatted into a structured PDF that can be previewed and shared externally.

The application uses Room Database for local data storage, ensuring an offline-first experience. From an architectural perspective, it follows a simplified MVVM pattern where the UI layer handles user interaction, ViewModels manage application logic, and the repository layer handles data operations. This structured approach ensures smooth performance, easy navigation, and scalability for future enhancements.

Chapter 4: Results and Analysis

4.1 Outcomes of Internship

The internship resulted in the successful development of a structured and functional Android application that demonstrates the core components of a real-world mobile system. The application was designed and implemented using Kotlin for application logic and XML for user interface design, following standard Android development practices. The successful execution of the project confirms that the application architecture, navigation flow, and data handling mechanisms have been correctly implemented.

One of the most significant outcomes of this internship is the ability to build a complete Android application from the initial setup phase to the final execution stage. This includes configuring the development environment, managing Gradle dependencies, designing user interfaces, implementing business logic, and testing the application on a real device. The successful generation of the APK file and its installation validate that the application is stable and operational.

The application developed includes multiple interconnected modules such as the home dashboard, customer management system, transaction ledger, invoice generation, and QR-based confirmation. Each module is implemented as a separate component, demonstrating an understanding of Android's modular architecture. The integration of these modules ensures smooth navigation and accurate data flow across different screens.

Another key outcome is the implementation of an offline-first system using Room Database, enabling the application to function without internet connectivity. The use of efficient UI components such as list-based views and structured layouts ensures responsive performance and scalability. Additionally, the internship provided practical exposure to debugging, dependency management, and performance optimization, contributing to a deeper understanding of real-world application development. Overall, the outcomes of this internship reflect the development of strong technical skills and the ability to design and implement a complete mobile application that addresses real-world problems effectively.

4.2 Performance Analysis

The performance of the application was evaluated based on key parameters such as responsiveness, stability, memory efficiency, and user interaction. The application performs smoothly during execution, with no significant delays or crashes observed during testing. Screen transitions between different modules such as home, customer list, ledger, and transaction entry are fast and seamless, ensuring a smooth user experience.

One of the primary factors contributing to performance is the efficient handling of data using Room Database. Since the application follows an offline-first approach, all transactions and customer data are stored locally, eliminating network latency and ensuring fast data retrieval.

The use of structured database queries and optimized data access methods enables quick updates and real-time balance calculations.

The application architecture also plays a key role in performance. A simplified MVVM pattern is used, where ViewModels such as LedgerViewModel, ProfileViewModel, and ConfirmationViewModel handle business logic and state management. This reduces

unnecessary UI recompositions and ensures that only relevant data updates are reflected on the screen, improving efficiency and responsiveness.

User interface performance is optimized using efficient UI components and structured layouts. List-based screens such as customer lists and transaction history are implemented using optimized list rendering techniques, ensuring smooth scrolling and minimal memory usage. The interface responds instantly to user actions such as adding transactions, confirming entries, and navigating between screens.

QR generation and validation processes are lightweight and executed quickly without affecting overall performance. Similarly, invoice generation using PDF libraries is handled efficiently without causing noticeable delays. Background operations such as file handling and data export are managed in a way that does not block the main UI thread.

The application was tested on a real Android device, where it demonstrated stable performance under continuous usage. No runtime crashes, memory leaks, or unexpected behavior were observed. The successful execution of Gradle builds and dependency integration further confirms the stability of the project setup.

Overall, the performance analysis indicates that the application is efficient, stable, and responsive. The design choices such as offline-first architecture, modular structure, and optimized data handling ensure that the application can scale effectively with additional features in future development phases

4.3 Challenges Faced

The development of the application involved several practical challenges during implementation. One of the primary challenges was handling offline data management effectively. Since the application follows an offline-first approach, ensuring that transactions, customer data, and balances were stored and updated correctly using Room Database required careful design. Maintaining data consistency and avoiding incorrect calculations during frequent updates was initially difficult.

Another challenge was planning for future migration to cloud synchronization. Although the core system is designed to work offline, integrating optional cloud backup using Firebase introduced complexity. Structuring the data model in a way that supports both local storage and future cloud sync without conflicts required additional planning.

Build-related issues were also a major challenge. Gradle build errors, dependency conflicts, and version mismatches frequently interrupted development. Identifying the root cause of these issues required careful analysis of error logs and understanding of dependency management.

Android Studio-related issues such as slow builds, emulator performance problems, and occasional IDE crashes affected development efficiency. Managing project configuration, clearing caches, and optimizing the development environment were necessary to maintain stability.

Time management was another challenge, as the project involved multiple stages including design, development, testing, and documentation. Balancing these tasks within the given timeline required proper planning and prioritization.

4.4 Solutions Implemented

The challenges encountered during development were addressed through structured approaches and appropriate technical solutions. For offline data handling, Room Database was implemented with well-defined entities and queries. Proper validation and update logic ensured that transaction data remained consistent and balances were calculated accurately.

To handle future cloud integration, the data layer was designed with scalability in mind. A repository pattern was used to abstract data sources, allowing seamless extension to cloud synchronization without affecting the core offline functionality.

Build-related issues were resolved by carefully analyzing Gradle error logs, updating dependencies, and maintaining compatibility between library versions. Regular project clean and rebuild operations helped stabilize the build process.

Android Studio performance issues were addressed by optimizing emulator settings, clearing IDE caches, and using real device testing where necessary. This improved development speed and reduced interruptions.

Time management challenges were handled by dividing the project into smaller modules and completing each phase step-by-step. This ensured steady progress and allowed focused development without overwhelming complexity.

Through these solutions, the application achieved stability, efficiency, and scalability, ensuring that it meets both current requirements and future expansion needs.

Chapter 5: Learning Outcomes

5.1 Technical Skills Gained

The internship significantly enhanced technical proficiency in Android application development, particularly in understanding how different components interact to form a complete and functional system. One of the primary skills developed was working with the Kotlin programming language. Through continuous implementation, concepts such as null safety, structured data handling, and clean coding practices were applied to build maintainable and scalable application logic.

Another important skill gained was designing user interfaces using XML and Material Design principles. Creating layouts that are both functional and user-friendly required careful attention to alignment, spacing, and responsiveness. The use of structured layout components and consistent design patterns improved the overall usability of the application, especially for users with minimal technical experience.

The internship also provided hands-on experience in implementing an offline-first data management system using Room Database. Designing entities, managing relationships, and performing efficient queries helped in understanding how local data storage works in real-world applications. Ensuring data consistency and real-time updates for transactions and balances strengthened practical database handling skills.

A key learning outcome was understanding application architecture using a simplified MVVM pattern. ViewModels were used to manage business logic and application state, while the repository layer handled data operations. This separation of concerns improved code organization, reduced dependency between components, and made the application easier to maintain and scale.

Additionally, the internship provided practical exposure to dependency management and build systems using Gradle. Resolving build errors, managing library dependencies, and generating APK files helped in understanding the complete development lifecycle. Experience with Android Studio tools, debugging using Logcat, and testing on real devices further strengthened overall development skills.

Overall, the internship helped in building a strong technical foundation in mobile application development, combining programming, UI design, database management, and architectural practices required for developing real-world Android applications

5.2 Soft Skills Developed

In addition to technical skills, the internship played a significant role in developing essential soft skills that are crucial for professional growth. One of the most important skills developed during this period is problem-solving ability. Developing an application involves encountering multiple issues such as build errors, runtime bugs, and logical inconsistencies. Identifying the root cause of these problems and implementing appropriate solutions required analytical thinking and a systematic approach.

Time management is another important skill that was developed during the internship. The project involved multiple stages, including planning, designing, coding, testing, and documentation. Managing these tasks within a limited timeframe required prioritization and

efficient use of available time. Breaking the project into smaller tasks and completing them step by step helped in maintaining steady progress.

The internship also improved self-learning ability. Since not all problems have direct solutions, it was necessary to explore documentation, experiment with different approaches, and learn new concepts independently. This ability to learn and adapt is essential in the field of software development, where technologies are constantly evolving.

Communication skills were also enhanced during this internship. Preparing presentations, explaining the project, and documenting the work required clear and structured communication. Being able to explain technical concepts in a simple and understandable manner is an important skill, especially during evaluations and team collaborations.

Another important soft skill developed is adaptability. During development, changes in requirements, unexpected errors, and new challenges required flexibility in approach. Adapting to these changes and finding solutions helped in improving overall efficiency and confidence in handling real-world scenarios.

5.3 Practical Exposure

The internship provided valuable practical exposure to real-world application development, bridging the gap between theoretical knowledge and practical implementation. Unlike academic exercises that focus on isolated concepts, this project required integrating multiple components such as UI, business logic, database handling, and system flow to build a complete and functional application. This helped in understanding how different layers of an application interact to deliver end-to-end functionality.

One of the key aspects of practical exposure was working with a complete development environment. Setting up the project in Android Studio, configuring dependencies, resolving Gradle build issues, and managing project structure provided hands-on experience with industry-standard tools. This also included understanding how to troubleshoot build failures and maintain a stable development setup.

Testing the application on a real device provided insight into real-world performance and usability. It helped in identifying issues related to UI responsiveness, data handling, and device compatibility. This experience highlighted the importance of optimizing applications for different hardware conditions and ensuring smooth operation under practical usage scenarios.

Another important area of exposure was implementing an offline-first system. Handling data locally using Room Database, ensuring consistency in transaction records, and maintaining correct balance calculations provided real experience in designing reliable systems without network dependency. This is a critical aspect in real-world applications where connectivity may not always be available.

The internship also provided exposure to application architecture and structured development practices. Implementing a modular approach using ViewModels and repository patterns helped in understanding how scalable and maintainable applications are built. This included managing state efficiently and separating UI logic from data handling.

Practical experience was also gained in handling file operations such as generating PDF invoices and managing local storage. This involved working with external libraries and ensuring that these operations do not affect application performance or user experience.

Additionally, debugging and problem-solving played a significant role throughout development. Using tools like Logcat to trace errors, identify issues, and test different scenarios improved analytical thinking and debugging efficiency. Handling real issues such as data inconsistencies, UI glitches, and dependency conflicts provided deeper insight into real-world development challenges.

The internship also introduced the use of AI-assisted tools for coding, debugging, and improving development productivity. These tools helped in understanding modern development workflows and how automation can assist in writing and optimizing code.

Overall, the internship provided a comprehensive practical learning experience by combining development, testing, debugging, and system design. It helped in building confidence to develop complete applications independently and laid a strong foundation for future work involving advanced features such as cloud integration and intelligent systems.

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

The internship provided a comprehensive learning experience by combining theoretical concepts with practical implementation in the domain of Android application development. Through this project, a functional mobile application titled “Namma-Santhe Ledger – Digital Khata App for Rural Vendors” was successfully designed and developed, demonstrating the ability to build a real-world system from the ground up. The project effectively integrates user interface design, application logic, and structured data handling to create a reliable and usable application.

One of the key achievements of this internship is the successful development of a multi-module Android application with proper navigation and user interaction. The implementation of core modules such as customer management, transaction ledger, invoice generation, and reporting reflects a clear understanding of application structure and real-world usability. The use of modular design principles ensures that the application is efficient, maintainable, and scalable.

The project highlights the importance of designing systems that work under real-world constraints. By implementing an offline-first approach using Room Database, the application ensures reliable operation without internet connectivity. Features such as quick transaction entry and structured data management improve usability and efficiency for small vendors.

Another important aspect of this internship is the development of problem-solving and debugging skills. Challenges such as handling offline data, managing dependencies, and resolving runtime issues were systematically addressed. These experiences contributed to a deeper understanding of application development and debugging practices.

The project also demonstrates the adoption of modern development practices, including the use of Kotlin, XML-based UI design, Material Design principles, and a structured architecture approach. Exposure to AI-assisted tools further improved productivity and understanding of modern workflows.

Overall, the internship successfully achieved its objective of providing hands-on experience in Android application development. It helped bridge the gap between academic knowledge and practical implementation, enabling the development of a complete and functional application. The knowledge and experience gained form a strong foundation for future work in mobile application development, cloud integration, and advanced system design.

6.2 Future Enhancements / Scope

Although the current version of the application successfully demonstrates the core functionality, there are several opportunities for further enhancement and development. One of the primary improvements planned is the integration of cloud-based data synchronization using Firebase. This will allow transaction data, customer records, and invoices to be securely backed up and accessed across multiple devices, improving data reliability and scalability.

Another important enhancement is the implementation of advanced analytics and reporting features. Future versions of the application can include detailed insights such as daily profit analysis, payment trends, and overdue tracking. These features will help vendors make better financial decisions and improve overall business management.

Multi-language support can be expanded beyond the current implementation to include additional regional languages. This will make the application more accessible to a wider range of users across different regions. Enhancing localization and improving user interface adaptability will further improve usability.

The application can also be extended to include voice-based interaction features. By integrating speech recognition and text-to-speech technologies, vendors can record transactions using voice commands and receive audio-based summaries. This will improve accessibility, especially for users with low literacy levels.

User interface improvements such as better visualizations, animations, and enhanced navigation can be incorporated to improve the overall user experience. Performance optimization techniques can also be applied to ensure smooth operation as the application scales and handles larger datasets.

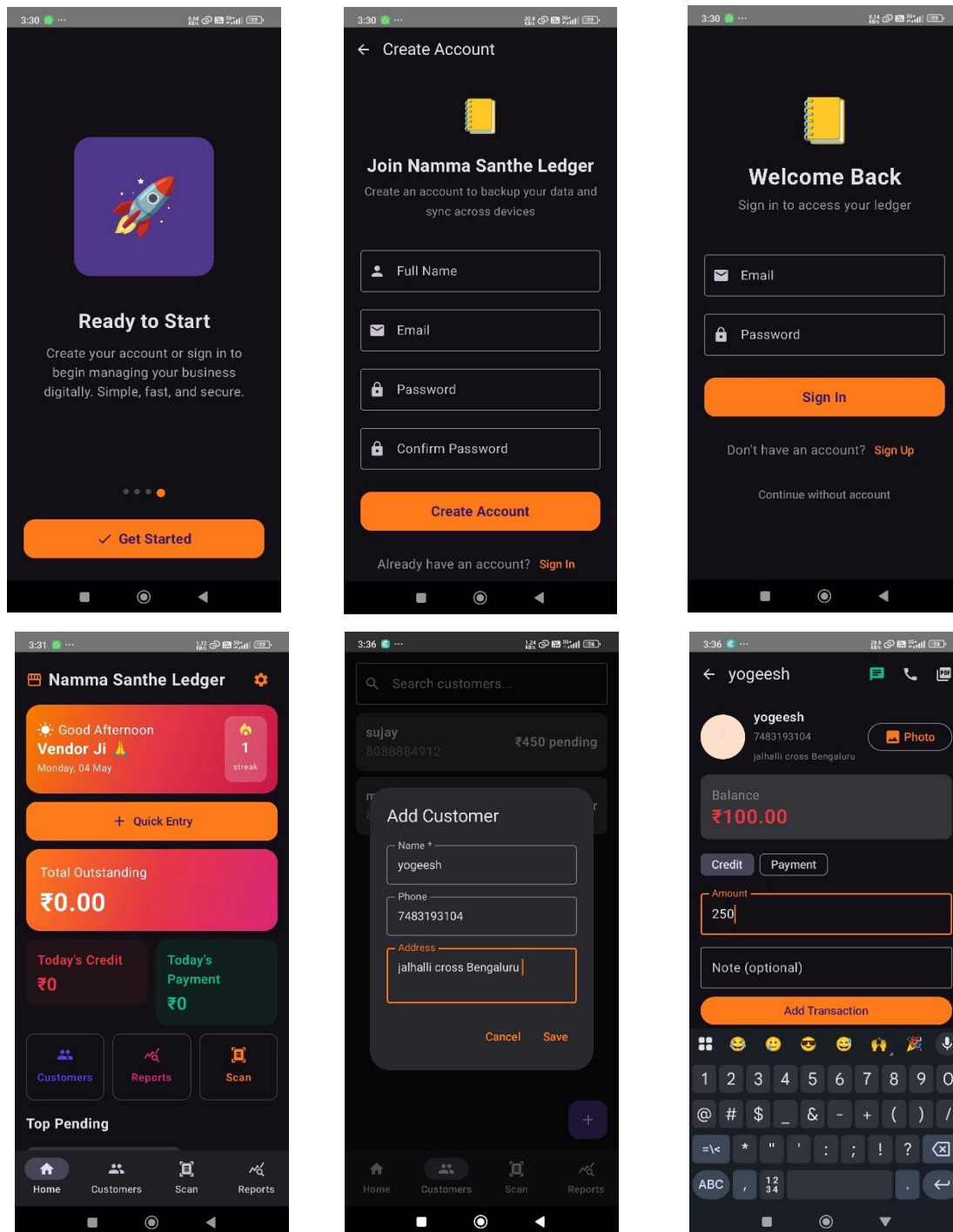
Security and data management can be further strengthened by enhancing authentication mechanisms and introducing role-based access control. Features such as data encryption and secure backups will make the application more reliable for real-world deployment.

Additionally, the application can be enhanced by integrating digital payment support such as UPI-based transaction recording and payment tracking. This will allow vendors to manage both cash and digital transactions within a single platform.

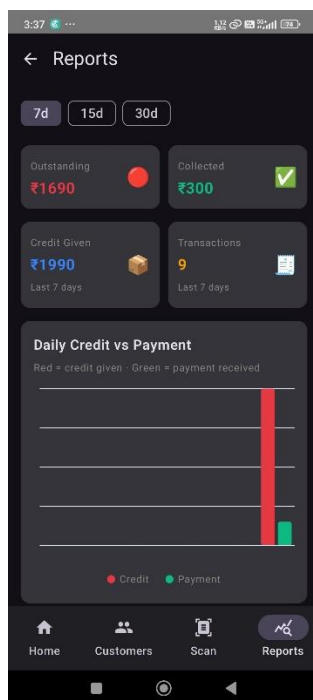
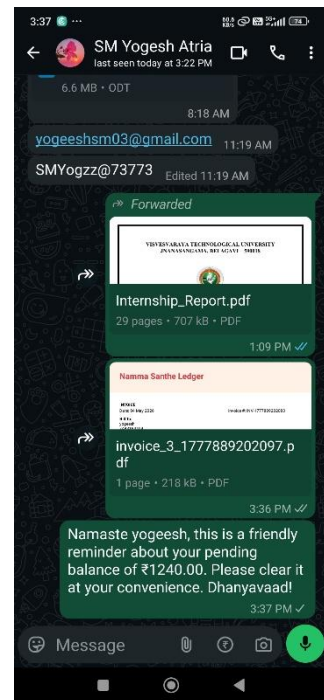
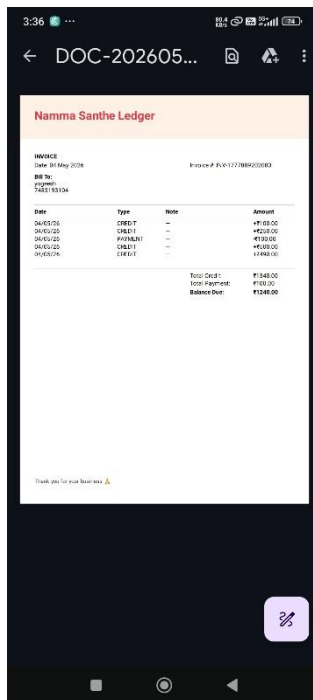
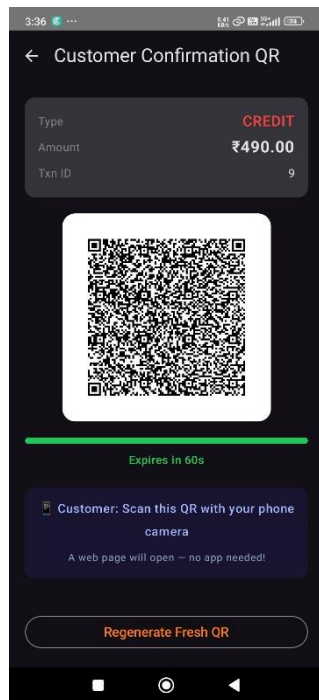
Overall, the future scope of the project is extensive, with multiple opportunities for improvement in terms of scalability, usability, and functionality. The current implementation provides a strong foundation, and with the integration of advanced features and cloud technologies, the application can evolve into a complete digital financial management solution for small vendors.

Appendices

Screenshots:



Namma-Santhe Ledger – Digital Khata App for Rural Vendors



Code:

Customer Ledger Screen

This screen handles transaction management for a specific customer, including balance calculation, QR confirmation, and photo proof.

```
// CustomerLedgerScreen.kt
package com.nammasanthe.ledger.ui.screens

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CustomerLedgerScreen(
    viewModel: LedgerViewModel,
    profileViewModel: ProfileViewModel,
    confirmationVm: ConfirmationViewModel,
    customerId: Long,
    onBack: () -> Unit,
    onShowQr: (Long) -> Unit,
    onTransactionClick: (Long) -> Unit
) {
    var customer by remember { mutableStateOf<Customer?>(null) }
    var transactions by remember {
        mutableStateOf<List<TxnEntity>>(emptyList()) }

    LaunchedEffect(customerId) {
        customer = viewModel.getCustomer(customerId)
        viewModel.customerTransactions(customerId).collectLatest {
            transactions = it
        }
    }

    val balance = transactions.sumOf {
        if (it.type == TxnType.CREDIT) it.amount else -it.amount
    }

    val scope = rememberCoroutineScope()
    var amount by remember { mutableStateOf("") }
    var note by remember { mutableStateOf("") }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(customer?.name ?: "Customer") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, null)
                    }
                }
            )
        }
    ) { padding ->
        Column(Modifier.padding(padding).padding(16.dp)) {

            Text("Balance: ₹${"%0.2f".format(balance)}")

            OutlinedTextField(
                value = amount,
                onValueChange = { amount = it },
                label = { Text("Amount") }
```

```

    )

    OutlinedTextField(
        value = note,
        onChange = { note = it },
        label = { Text("Note") }
    )

    Button(onClick = {
        val amt = amount.toDoubleOrNull() ?: return@Button
        scope.launch {
            viewModel.addTransaction(customerId, amt, note)
        }
    }) {
        Text("Add Transaction")
    }

    LazyColumn {
        items(transactions) { txn ->
            Text(
                "${txn.type} ₹${txn.amount}",
                modifier = Modifier.clickable {
                    onTransactionClick(txn.id)
                }
            )
        }
    }
}
}
}
}
}

```

Home Screen

Displays dashboard metrics like outstanding balance and quick navigation.

```

package com.nammasanthe.ledger.ui.screens

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.Chat
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import com.nammasanthe.ledger.util.ContactActions
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.nammasanthe.ledger.ui.components.GradientCard
import com.nammasanthe.ledger.ui.components.GreetingCard
import com.nammasanthe.ledger.ui.components.SectionHeader
import com.nammasanthe.ledger.ui.components.StatTile
import com.nammasanthe.ledger.viewmodel.ProfileViewModel

```

```
import com.nammasanthe.ledger.ui.theme.*
import com.nammasanthe.ledger.viewmodel.LedgerViewModel

@Composable
fun HomeScreen(
    viewModel: LedgerViewModel,
    profileViewModel: ProfileViewModel,
    onQuickEntry: () -> Unit,
    onCustomers: () -> Unit,
    onReports: () -> Unit,
    onScanner: () -> Unit,
    onProfile: () -> Unit,
    onCustomer: (Long) -> Unit
) {
    val profile by profileViewModel.profile.collectAsStateWithLifecycle()
    val outstanding by viewModel.outstanding.collectAsStateWithLifecycle()
    val balances by viewModel.balances.collectAsStateWithLifecycle()
    val recent by viewModel.recent.collectAsStateWithLifecycle()
    val customers by viewModel.customers.collectAsStateWithLifecycle()
    val ctx = LocalContext.current

    val today = java.util.Calendar.getInstance().apply {
        set(java.util.Calendar.HOUR_OF_DAY, 0);
    }
    set(java.util.Calendar.MINUTE, 0)
    set(java.util.Calendar.SECOND, 0);
    set(java.util.Calendar.MILLISECOND, 0)
    }.timeInMillis
    val todayCredit = recent.filter { it.date >= today && it.type.name ==
"CREDIT" }.sumOf { it.amount }
    val todayPayment = recent.filter { it.date >= today && it.type.name ==
"PAYMENT" }.sumOf { it.amount }

    Column(

Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(16
.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Icon(Icons.Default.Storefront, null, tint = Saffron)
            Spacer(Modifier.width(8.dp))
            Text("Namma Santhe Ledger", fontSize = 22.sp, fontWeight =
FontWeight.Bold, modifier = Modifier.weight(1f))
            IconButton(onClick = onProfile) { Icon(Icons.Default.Settings,
"Profile", tint = Saffron) }
        }
        GreetingCard(businessName = profile.businessName.ifBlank {
profile.ownerName })

        Button(
            onClick = onQuickEntry,
            modifier = Modifier.fillMaxWidth().height(56.dp),
            shape = RoundedCornerShape(16.dp)
        ) {
            Icon(Icons.Default.Add, null); Spacer(Modifier.width(8.dp))
            Text("Quick Entry", fontWeight = FontWeight.SemiBold)
        }

        GradientCard(brush = PrimaryGradient) {
            Text("Total Outstanding", color = Color.White)
            Spacer(Modifier.height(8.dp))
            Text("₹${"%0.2f".format(outstanding)}", color = Color.White,
```

```

        fontSize = 36.sp, fontWeight = FontWeight.Bold)
    }

    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        StatTile("Today's Credit", "₹${"%0f".format(todayCredit)}",
        Crimson, Modifier.weight(1f))
        StatTile("Today's Payment", "₹${"%0f".format(todayPayment)}",
        Emerald500, Modifier.weight(1f))
    }

    Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
        QuickAction("Customers", Icons.Default.Group, Indigo600,
        Modifier.weight(1f), onCustomers)
        QuickAction("Reports", Icons.Default.QueryStats, Pink600,
        Modifier.weight(1f), onReports)
        QuickAction("Scan", Icons.Default.DocumentScanner, Saffron,
        Modifier.weight(1f), onScanner)
    }

    SectionHeader("Top Pending")
    val top = balances.filter { it.balance > 0 }.sortedByDescending {
    it.balance }.take(3)
    if (top.isEmpty()) {
        Card { Box(Modifier.padding(16.dp)) { Text("All clear – no dues
        🍹") } }
    } else {
        top.forEach { b ->
            Card(modifier = Modifier.fillMaxWidth()) {
                Row(Modifier.padding(12.dp), verticalAlignment =
                Alignment.CenterVertically) {
                    Column(
                        Modifier.weight(1f).padding(end = 8.dp)
                    ) {
                        Text(b.name, fontWeight = FontWeight.SemiBold)
                        Text(b.phone, color = Color.Gray, fontSize =
                        12.sp)
                        Text("₹${"%0f".format(b.balance)} pending",
                        color = Crimson, fontWeight = FontWeight.Bold)
                    }
                    IconButton(onClick = {
                        ContactActions.whatsappReminder(ctx, b.name,
                        b.phone, b.balance)
                    }) { Icon(Icons.AutoMirrored.Filled.Chat,
                    contentDescription = "WhatsApp", tint = Emerald500) }
                    IconButton(onClick = { ContactActions.call(ctx,
                    b.phone) }) {
                        Icon(Icons.Default.Call, contentDescription =
                        "Call", tint = Indigo600)
                    }
                    IconButton(onClick = { onCustomer(b.customerId) })
                    {
                        Icon(Icons.Default.ChevronRight,
                        contentDescription = "Open")
                    }
                }
            }
        }
    }

    SectionHeader("Recent Transactions")
    if (recent.isEmpty()) {
        Card { Box(Modifier.padding(16.dp)) { Text("No transactions

```



```

        yet") } }
        } else {
            recent.take(5).forEach { t ->
                val name = customers.firstOrNull { it.id == t.customerId
            }?.name ?: "-"
            Card(Modifier.fillMaxWidth()) {
                Row(Modifier.padding(14.dp), verticalAlignment =
                Alignment.CenterVertically) {
                    Column(Modifier.weight(1f)) {
                        Text(name, fontWeight = FontWeight.SemiBold)
                        Text(t.type.name, color = Color.Gray, fontSize
                        = 12.sp)
                    }
                    val color = if (t.type.name == "CREDIT") Crimson
                else Emerald500
                    Text(
                        "${if (t.type.name == "CREDIT") "+" else "-"}
                        ₹${"%.0f".format(t.amount)}",
                        color = color,
                        fontWeight = FontWeight.Bold
                    )
                }
            }
        }
    }
}

@Composable
private fun QuickAction(
    label: String,
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    accent: Color,
    modifier: Modifier,
    onClick: () -> Unit
) {
    OutlinedCard(onClick = onClick, modifier = modifier.height(96.dp)) {
        Column(
            Modifier.fillMaxSize().padding(8.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Icon(icon, null, tint = accent)
            Spacer(Modifier.height(6.dp))
            Text(label, color = accent, fontSize = 13.sp, fontWeight =
            FontWeight.SemiBold)
        }
    }
}

```

Signup Screen

The Signup Screen allows new users to register by entering their details, validates input, and creates an account using Firebase Authentication.

```

import android.widget.Toast
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack

```

```
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Person
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.unit.dp
import com.nammasanthe.ledger.sync.FirebaseAuthManager
import kotlin.coroutines.launch

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SignupScreen(
    onSignupSuccess: () -> Unit,
    onNavigateToLogin: () -> Unit,
    onBack: () -> Unit
) {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val authManager = remember { FirebaseAuthManager.getInstance(context) }

    var name by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var confirmPassword by remember { mutableStateOf("") }

    var isLoading by remember { mutableStateOf(false) }
    var error by remember { mutableStateOf<String?>(null) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Create Account") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, null)
                    }
                }
            )
        }
    ) { padding ->
        Column(
            modifier = Modifier
                .padding(padding)
                .padding(24.dp)
                .fillMaxSize()
                .verticalScroll(rememberScrollState()),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            OutlinedTextField(
                value = name,
                onChange = { name = it },
                label = { Text("Full Name") },
                leadingIcon = { Icon(Icons.Default.Person, null) },
                modifier = Modifier.fillMaxWidth()
            )
        }
    }
}
```

```

        Spacer(Modifier.height(12.dp))

        OutlinedTextField(
            value = email,
            onChange = { email = it },
            label = { Text("Email") },
            leadingIcon = { Icon(Icons.Default.Email, null) },
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(12.dp))

        OutlinedTextField(
            value = password,
            onChange = { password = it },
            label = { Text("Password") },
            leadingIcon = { Icon(Icons.Default.Lock, null) },
            visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(12.dp))

        OutlinedTextField(
            value = confirmPassword,
            onChange = { confirmPassword = it },
            label = { Text("Confirm Password") },
            leadingIcon = { Icon(Icons.Default.Lock, null) },
            visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(20.dp))

        error?.let {
            Text(it, color = MaterialTheme.colorScheme.error)
            Spacer(Modifier.height(8.dp))
        }

        Button(
            onClick = {
                when {
                    name.isBlank() -> error = "Enter name"
                    email.isBlank() -> error = "Enter email"
                    password.length < 6 -> error = "Password too short"
                    password != confirmPassword -> error = "Passwords
do not match"
                }
                else -> {
                    scope.launch {
                        isLoading = true
                        error = null
                        val result = authManager.signUp(email,
password, name)

                        isLoading = false

                        result.onSuccess {
                            Toast.makeText(context, "Signup
Successful", Toast.LENGTH_SHORT).show()
                            onSignupSuccess()
                        }.onFailure {
                            error = it.message
                        }
                    }
                }
            }
        )
    }
}

```

```

        }
    }
    },
    modifier = Modifier.fillMaxWidth(),
    enabled = !isLoading
) {
    Text(if (isLoading) "Creating..." else "Sign Up")
}

Spacer(Modifier.height(12.dp))

TextButton(onClick = onNavigateToLogin) {
    Text("Already have an account? Login")
}
}
}
}

```

Login Screen

Handles authentication using Firebase.

```

package com.nammasanthe.ledger.ui.screens

import android.widget.Toast
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.nammasanthe.ledger.sync.FirebaseAuthManager
import kotlinx.coroutines.launch

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LoginScreen(
    onLoginSuccess: () -> Unit,
    onNavigateToSignup: () -> Unit,
    onSkip: () -> Unit
) {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val authManager = remember { FirebaseAuthManager.getInstance(context) }

    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var isLoading by remember { mutableStateOf(false) }
    var emailError by remember { mutableStateOf<String?>(null) }

```

```

var passwordError by remember { mutableStateOf<String?>(null) }

Scaffold(
    modifier = Modifier.fillMaxSize()
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .padding(horizontal = 24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // App icon/title
        Box(
            modifier = Modifier
                .size(100.dp)
                .padding(bottom = 24.dp),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = "■",
                fontSize = 64.sp
            )
        }

        Text(
            text = "Welcome Back",
            fontSize = 32.sp,
            fontWeight = FontWeight.Bold,
            textAlign = TextAlign.Center
        )

        Text(
            text = "Sign in to access your ledger",
            fontSize = 16.sp,
            color = MaterialTheme.colorScheme.onBackground.copy(alpha =
0.6f),
            textAlign = TextAlign.Center,
            modifier = Modifier.padding(top = 8.dp, bottom = 48.dp)
        )

        // Email field
        OutlinedTextField(
            value = email,
            onValueChange = {
                email = it
                emailError = null
            },
            modifier = Modifier.fillMaxWidth(),
            label = { Text("Email") },
            leadingIcon = { Icon(Icons.Default.Email, null) },
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Email,
                imeAction = ImeAction.Next
            ),
            isError = emailError != null,
            supportingText = emailError?.let { err -> { Text(err) } },
            singleLine = true
        )

        Spacer(modifier = Modifier.height(16.dp))
    }
}

```

```

        // Password field
        OutlinedTextField(
            value = password,
            onChange = {
                password = it
                passwordError = null
            },
            modifier = Modifier.fillMaxWidth(),
            label = { Text("Password") },
            leadingIcon = { Icon(Icons.Default.Lock, null) },
            visualTransformation = PasswordVisualTransformation(),
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Password,
                imeAction = ImeAction.Done
            ),
            isError = passwordError != null,
            supportingText = passwordError?.let { err -> { Text(err) } }
        ),

        singleLine = true
    )

    Spacer(modifier = Modifier.height(32.dp))

    // Sign In button
    Button(
        onClick = {
            // Validate inputs
            var hasError = false
            if (email.isBlank()) {
                emailError = "Email is required"
                hasError = true
            }
            if (password.isBlank()) {
                passwordError = "Password is required"
                hasError = true
            } else if (password.length < 6) {
                passwordError = "Password must be at least 6
characters"
                hasError = true
            }

            if (hasError) return@Button

            isLoading = true
            scope.launch {
                val result = authManager.signIn(email, password)
                isLoading = false
                result.onSuccess {
                    Toast.makeText(context, "Welcome back!",
Toast.LENGTH_SHORT).show()
                    onLoginSuccess()
                }.onFailure { e ->
                    Toast.makeText(
                        context,
                        e.message ?: "Login failed",
                        Toast.LENGTH_LONG
                    ).show()
                }
            }
        },
        modifier = Modifier

```

```

        .fillMaxWidth()
        .height(56.dp),
        shape = RoundedCornerShape(16.dp),
        enabled = !isLoading
    ) {
        if (isLoading) {
            CircularProgressIndicator(
                modifier = Modifier.size(24.dp),
                color = MaterialTheme.colorScheme.onPrimary
            )
        } else {
            Text(
                text = "Sign In",
                fontSize = 18.sp,
                fontWeight = FontWeight.Bold
            )
        }
    }

    Spacer(modifier = Modifier.height(24.dp))

    // Sign up link
    Row(
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = "Don't have an account?",
            color =
MaterialTheme.colorScheme.onBackground.copy(alpha = 0.6f)
        )
        TextButton(onClick = onNavigateToSignup) {
            Text("Sign Up")
        }
    }

    // Skip option
    TextButton(
        onClick = onSkip,
        modifier = Modifier.padding(top = 16.dp)
    ) {
        Text(
            text = "Continue without account",
            color =
MaterialTheme.colorScheme.onBackground.copy(alpha = 0.5f)
        )
    }
}
}

```

QR Display Screen

Displays QR for transaction confirmation with expiry timer.

```

package com.nammasanthe.ledger.ui.screens

import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition

```

```

import androidx.compose.animation.core.tween
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.scale
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.nammasanthe.ledger.security.TrustLevel
import com.nammasanthe.ledger.ui.components.TrustBadge
import com.nammasanthe.ledger.viewmodel.ConfirmationViewModel
import com.nammasanthe.ledger.viewmodel.QrDisplayState
import kotlin.coroutines.delay

private const val QR_TTL_SECONDS = 60

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun QrDisplayScreen(
    txnId          : Long,
    confirmationVm  : ConfirmationViewModel,
    onBack          : () -> Unit
) {
    val qrState by confirmationVm.qrState.collectAsStateWithLifecycle()

    // Request QR generation on first composition
    LaunchedEffect(txnId) {
        confirmationVm.requestQrForTxnId(txnId)
    }

    // Countdown timer – only runs while QR is showing
    var secondsLeft by remember { mutableIntStateOf(QR_TTL_SECONDS) }
    LaunchedEffect(qrState) {
        if (qrState is QrDisplayState.Ready) {
            secondsLeft = QR_TTL_SECONDS
            while (secondsLeft > 0) {
                delay(1_000)
                secondsLeft--
            }
            confirmationVm.markQrExpired()
        }
    }

    val timerFraction by animateFloatAsState(
        targetValue = secondsLeft / QR_TTL_SECONDS.toFloat(),
        animationSpec = tween(durationMillis = 800),
        label = "timerFraction"
    )

```

```

)

DisposableEffect(Unit) { onDispose { confirmationVm.clearQr() } }

Scaffold(
  topBar = {
    TopAppBar(
      title = { Text("Customer Confirmation QR") },
      navigationIcon = {
        IconButton(onClick = onBackPressed) {
          Icon(Icons.AutoMirrored.Filled.ArrowBack,
contentDescription = null)
        }
      }
    )
  }
) { padding ->
  Column(
    modifier = Modifier
      .padding(padding)
      .fillMaxSize()
      .padding(20.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.spacedBy(20.dp)
  ) {
    when (val s = qrState) {
      QrDisplayState.Idle -> {
        Spacer(Modifier.weight(1f))
        CircularProgressIndicator()
        Text("Generating QR...", color = Color.Gray)
        Spacer(Modifier.weight(1f))
      }

      QrDisplayState.Expired -> {
        Spacer(Modifier.weight(1f))
        Text("❑", fontSize = 52.sp)
        Text(
          "QR Expired",
          fontSize = 22.sp,
          fontWeight = FontWeight.Bold
        )
        Text(
          "The 60-second window has closed.\nGenerate a fresh
QR to confirm.",
          textAlign = TextAlign.Center,
          color = Color.Gray
        )
        Spacer(Modifier.height(8.dp))
        Button(
          onClick = {
confirmationVm.requestQrForTxnId(txnId) },
          modifier = Modifier.fillMaxWidth()
        ) { Text("Regenerate QR") }
        Spacer(Modifier.weight(1f))
      }

      // — Firebase confirmed!

      is QrDisplayState.ConfirmedViaFirebase -> {
        Spacer(Modifier.weight(1f))

        val (icon, msg, color) = when (s.trustLevel) {

```

```

        TrustLevel.VERIFIED    -> Triple("✔", "Customer
Confirmed!", Color(0xFF22C55E))
        TrustLevel.SUSPICIOUS -> Triple("⚠", "Confirmed
with Warning", Color(0xFFFF59E0B))
        TrustLevel.UNVERIFIED -> Triple("❌", "Stored as
Unverified", Color(0xFF64748B))
    }
    Text(icon, fontSize = 64.sp)
    Spacer(Modifier.height(8.dp))
    Text(msg, fontWeight = FontWeight.Bold, fontSize =
22.sp, color = color)
    TrustBadge(s.trustLevel)

    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = Color(0xFF0F1F0F)
        )
    ) {
        Column(Modifier.padding(16.dp)) {
            Row(
                Modifier.fillMaxWidth(),
                horizontalArrangement =
Arrangement.SpaceBetween
            ) {
                Text("Amount", color = Color.Gray, fontSize
= 13.sp)

                Text(
                    "₹${"%0.2f".format(s.payload.amount)}",
                    fontWeight = FontWeight.Bold,
                    fontSize = 18.sp,
                    color = Color.White
                )
            }
            Spacer(Modifier.height(8.dp))
            Row(
                Modifier.fillMaxWidth(),
                horizontalArrangement =
Arrangement.SpaceBetween
            ) {
                Text("Source", color = Color.Gray, fontSize
= 13.sp)

                Text(
                    "Customer Web Confirmation",
                    fontSize = 13.sp,
                    color = Color(0xFF22C55E)
                )
            }
        }
    }

    Text(
        "The customer confirmed this transaction\nvia the
QR code web link.",
        textAlign = TextAlign.Center,
        color = Color.Gray,
        fontSize = 13.sp
    )

    Spacer(Modifier.weight(1f))
    Button(

```

```

        onClick = onBackPressed,
        modifier = Modifier.fillMaxWidth()
    ) { Text("Done") }
}

// — Firebase rejected

is QrDisplayState.RejectedViaFirebase -> {
    Spacer(Modifier.weight(1f))
    Text("❌", fontSize = 64.sp)
    Spacer(Modifier.height(8.dp))
    Text(
        "Customer Rejected",
        fontWeight = FontWeight.Bold,
        fontSize = 22.sp,
        color = Color(0xFFEF4444)
    )
    Text(
        "The customer rejected this transaction.\nPlease
verify the details with them.",
        textAlign = TextAlign.Center,
        color = Color.Gray,
        fontSize = 13.sp
    )
    Spacer(Modifier.weight(1f))
    Button(
        onClick = onBackPressed,
        modifier = Modifier.fillMaxWidth()
    ) { Text("Go Back") }
}

// — Waiting for Firebase (shown after countdown)

is QrDisplayState.WaitingForConfirmation -> {
    Spacer(Modifier.weight(1f))

    // Pulsing animation
    val infiniteTransition =
rememberInfiniteTransition(label = "pulse")
    val scale by infiniteTransition.animateFloat(
        initialValue = 0.9f,
        targetValue = 1.1f,
        animationSpec = infiniteRepeatable(
            animation = tween(1000, easing =
LinearEasing),
            repeatMode = RepeatMode.Reverse
        ),
        label = "pulseScale"
    )

    Box(
        modifier = Modifier
            .size(80.dp)
            .scale(scale)
            .clip(CircleShape)
            .background(
                Brush.radialGradient(
                    colors = listOf(Color(0xFF6366F1),
Color(0xFF818CF8))
                )
            ),
        contentAlignment = Alignment.Center
    )
}

```

```

    ) {
        Text("₹", fontSize = 36.sp)
    }

    Text(
        "Waiting for Customer...",
        fontWeight = FontWeight.Bold,
        fontSize = 20.sp
    )
    Text(
        "The QR code link has been shared.\nWaiting for the
customer to confirm.",
        textAlign = TextAlign.Center,
        color = Color.Gray,
        fontSize = 13.sp
    )

    CircularProgressIndicator(
        modifier = Modifier.size(32.dp),
        strokeWidth = 3.dp,
        color = Color(0xFF6366F1)
    )

    Spacer(Modifier.weight(1f))
}

// — QR Ready (main state)
is QrDisplayState.Ready -> {
    // — Transaction info

    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {
            Row(
                Modifier.fillMaxWidth(),
                horizontalArrangement =
Arrangement.SpaceBetween
            ) {
                Text("Type", color = Color.Gray, fontSize
= 12.sp)

                Text(
                    s.payload.type,
                    fontWeight = FontWeight.SemiBold,
                    color = if (s.payload.type ==
"CREDIT")
                        Color(0xFFEF4444) else
Color(0xFF22C55E)
                )
            }
            Row(
                Modifier
                    .fillMaxWidth()
                    .padding(top = 4.dp),
                horizontalArrangement =
Arrangement.SpaceBetween
            ) {
                Text("Amount", color = Color.Gray, fontSize
= 12.sp)

                Text(
                    "₹${"%0.2f".format(s.payload.amount)}",
                    fontWeight = FontWeight.Bold,
                    fontSize = 18.sp

```

```

        )
    }
    Row(
        Modifier
            .fillMaxWidth()
            .padding(top = 4.dp),
        horizontalArrangement =
Arrangement.SpaceBetween
    ) {
        Text("Txn ID", color = Color.Gray, fontSize
= 12.sp)
        Text(s.payload.txnId, fontSize = 12.sp)
    }
}

// — QR bitmap
Box(
    modifier = Modifier
        .size(260.dp)
        .clip(RoundedCornerShape(16.dp))
        .background(Color.White)
        .padding(12.dp),
    contentAlignment = Alignment.Center
) {
    Image(
        bitmap = s.bitmap.asImageBitmap(),
        contentDescription = "Confirmation QR code",
        modifier = Modifier.fillMaxSize()
    )
}

// — Countdown bar
Column(
    Modifier.fillMaxWidth(),
    horizontalAlignment = Alignment.CenterHorizontally
) {
    val barColor = when {
        secondsLeft > 30 -> Color(0xFF22C55E)
        secondsLeft > 10 -> Color(0xFFFF59E0B)
        else -> Color(0xFFEF4444)
    }
    LinearProgressIndicator(
        progress = { timerFraction },
        modifier = Modifier
            .fillMaxWidth()
            .height(8.dp)
            .clip(RoundedCornerShape(4.dp)),
        color = barColor,
        trackColor = barColor.copy(alpha =
0.2f),
        strokeCap = StrokeCap.Round
    )
    Spacer(Modifier.height(6.dp))
    Text(
        "Expires in ${secondsLeft}s",
        fontSize = 12.sp,
        color = barColor,
        fontWeight = FontWeight.SemiBold
    )
}

```

```
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.StrokeJoin
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.nammasanthe.ledger.data.dao.CustomerBalance
import com.nammasanthe.ledger.data.entity.TxnType
import com.nammasanthe.ledger.ui.theme.Crimson
import com.nammasanthe.ledger.ui.theme.Emerald500
import com.nammasanthe.ledger.viewmodel.LedgerViewModel
import kotlin.math.abs

// — palette
private val Blue      = Color(0xFF3B82F6)
private val Amber     = Color(0xFFFF59E0B)
private val Grid      = Color(0xFFE2E8F0)
private val ZeroRef   = Color(0xFF94A3B8)

// — per-day bucket
private data class DayData(
    val timestamp: Long,
    val credit: Double,
    val payment: Double
) {
    val net: Double get() = credit - payment
}

//
//
// Screen
//
//
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReportsScreen(viewModel: LedgerViewModel, onBack: () -> Unit) {

    var periodDays by remember { mutableIntStateOf(7) }
    val since = remember(periodDays) {
        System.currentTimeMillis() - periodDays.toLong() * 86_400_000L
    }

    // collect transactions for the chosen period
    val txnsFlow = remember(since) { viewModel.transactionsSince(since) }
    val allTxns by txnsFlow.collectAsStateWithLifecycle(emptyList())

    val outstanding by viewModel.outstanding.collectAsStateWithLifecycle()
```

```

    val balances      by viewModel.balances.collectAsStateWithLifecycle()

    // bucket into days
    val dailyData = remember(allTxns, periodDays, since) {
        (0 until periodDays).map { offset ->
            val dayStart = since + offset.toLong() * 86_400_000L
            val dayEnd   = dayStart + 86_400_000L
            val slice     = allTxns.filter { it.date in dayStart until
dayEnd }
            DayData(
                timestamp = dayStart,
                credit     = slice.filter { it.type == TxnType.CREDIT
}.sumOf { it.amount },
                payment    = slice.filter { it.type == TxnType.PAYMENT
}.sumOf { it.amount }
            )
        }
    }

    val totalCredit = dailyData.sumOf { it.credit }
    val totalPayment = dailyData.sumOf { it.payment }
    val txnCount     = allTxns.size
    val collRate     = if (outstanding + totalPayment > 0)
        totalPayment / (outstanding + totalPayment) * 100 else 0.0
    val topDebtors   = balances.filter { it.balance > 0 }.take(6)

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Reports") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack,
contentDescription = null)
                    }
                }
            )
        }
    ) { padding ->
        LazyColumn(
            modifier                = Modifier.padding(padding).fillMaxSize(),
            contentPadding          = PaddingValues(horizontal = 16.dp,
vertical = 12.dp),
            verticalArrangement     = Arrangement.spacedBy(16.dp)
        ) {

            // — Period selector


---


            item {
                PeriodSelector(selected = periodDays, onSelect = {
periodDays = it })
            }

            // — KPI row 1


---


            item {
                Row(Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(10.dp)) {
                    KpiCard(
                        label = "Outstanding",
                        value = "₹${"%0f".format(outstanding)}",
                        color = Crimson,

```

```

        icon = "●",
        mod = Modifier.weight(1f)
    )
    KpiCard(
        label = "Collected",
        value = "₹${"%0f".format(totalPayment)}",
        color = Emerald500,
        icon = "✓",
        mod = Modifier.weight(1f)
    )
    }
}

// — KPI row 2

item {
    Row(Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(10.dp)) {
        KpiCard(
            label = "Credit Given",
            value = "₹${"%0f".format(totalCredit)}",
            color = Blue,
            icon = "₹",
            mod = Modifier.weight(1f),
            sub = "Last $periodDays days"
        )
        KpiCard(
            label = "Transactions",
            value = "$txnCount",
            color = Amber,
            icon = "□",
            mod = Modifier.weight(1f),
            sub = "Last $periodDays days"
        )
    }
}

// — Bar chart

item {
    ReportCard(
        title = "Daily Credit vs Payment",
        subtitle = "Red = credit given · Green = payment
received"
    ) {
        val isEmpty = dailyData.all { it.credit == 0.0 &&
it.payment == 0.0 }
        if (isEmpty) EmptyMsg()
        else BarChart(dailyData = dailyData, periodDays
= periodDays)
    }
}

// — Line chart

item {
    ReportCard(
        title = "Daily Net Position",
        subtitle = "Credit - Payment per day (above zero = more
credit given)"
    ) {

```

```

        val isEmpty = dailyData.all { it.net == 0.0 }
        if (isEmpty) EmptyMsg()
        else NetLineChart(dailyData = dailyData)
    }
}

// — Donut chart

item {
    ReportCard(title = "Outstanding vs Collected") {
        DonutSection(
            outstanding = outstanding,
            collected = totalPayment,
            rate = collRate
        )
    }
}

// — Top debtors

item {
    ReportCard(
        title = "Top Debtors",
        subtitle = "Customers with highest pending balance"
    ) {
        if (topDebtors.isEmpty())
            Text(
                "🔔 No pending dues!",
                modifier =
Modifier.fillMaxWidth().padding(vertical = 16.dp),
                textAlign = TextAlign.Center,
                color = Color.Gray
            )
        else
            TopDebtorsChart(balances = topDebtors)
    }
}

// — Summary table

item {
    SummaryCard(
        totalCredit = totalCredit,
        totalPayment = totalPayment,
        txnCount = txnCount,
        periodDays = periodDays,
        dailyData = dailyData
    )
}

item { Spacer(Modifier.height(16.dp)) }
}

}

//

```

```

@Composable
private fun PeriodSelector(selected: Int, onSelect: (Int) -> Unit) {
    Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        listOf(7, 15, 30).forEach { d ->
            FilterChip(
                selected = (selected == d),
                onClick = { onSelect(d) },
                label = { Text("${d}d") }
            )
        }
    }
}

//
=====
// KPI card
//
=====
@Composable
private fun KpiCard(
    label : String,
    value : String,
    color : Color,
    icon : String,
    mod : Modifier = Modifier,
    sub : String? = null
) {
    Card(mod) {
        Row(
            modifier = Modifier.padding(12.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Column(Modifier.weight(1f)) {
                Text(label, fontSize = 11.sp, color = Color.Gray)
                Text(value, fontSize = 17.sp, fontWeight = FontWeight.Bold,
color = color)
                if (sub != null) Text(sub, fontSize = 10.sp, color =
Color.Gray)
            }
            Text(icon, fontSize = 22.sp)
        }
    }
}

//
=====
// Report card wrapper
//
=====
@Composable
private fun ReportCard(
    title : String,
    subtitle : String? = null,
    content : @Composable ColumnScope.() -> Unit
) {
    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {

```

```

        Text(title, fontWeight = FontWeight.SemiBold, fontSize = 15.sp)
        if (subtitle != null)
            Text(subtitle, fontSize = 11.sp, color = Color.Gray)
        Spacer(Modifier.height(12.dp))
        content()
    }
}

@Composable
private fun EmptyMsg() {
    Text(
        "No data for this period",
        modifier = Modifier.fillMaxWidth().padding(vertical = 20.dp),
        textAlign = TextAlign.Center,
        color = Color.Gray,
        fontSize = 13.sp
    )
}

//
=====
// Bar chart (side-by-side credit / payment per day)
//
=====

@Composable
private fun BarChart(dailyData: List<DayData>, periodDays: Int) {
    val crimson = Crimson
    val emerald = Emerald500

    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(220.dp)
    ) {
        val padLeft = 8.dp.toPx()
        val padBottom = 24.dp.toPx()
        val chartW = size.width - padLeft
        val chartH = size.height - padBottom

        val maxVal = (dailyData.flatMap { listOf(it.credit, it.payment) }
            .maxOrNull() ?: 1.0).coerceAtLeast(1.0)

        val n = dailyData.size
        val groupW = chartW / n
        val barPad = groupW * 0.08f
        val barW = (groupW - barPad * 3f) / 2f

        // grid lines
        val gridSteps = 4
        repeat(gridSteps + 1) { i ->
            val y = chartH * (1f - i.toFloat() / gridSteps)
            drawLine(Grid, Offset(padLeft, y), Offset(size.width, y),
                strokeWidth = 1.dp.toPx())
        }

        // bars
        dailyData.forEachIndexed { idx, day ->
            val xBase = padLeft + idx * groupW + barPad

```

```

        val cH = (day.credit / maxVal).toFloat() * chartH
        val pH = (day.payment / maxVal).toFloat() * chartH

        if (cH > 1f) {
            drawRoundRect(
                color = crimson,
                topLeft = Offset(xBase, chartH - cH),
                size = Size(barW, cH),
                cornerRadius = CornerRadius(3.dp.toPx())
            )
        }
        if (pH > 1f) {
            drawRoundRect(
                color = emerald,
                topLeft = Offset(xBase + barW + barPad, chartH -
pH),
                size = Size(barW, pH),
                cornerRadius = CornerRadius(3.dp.toPx())
            )
        }
    }

    // baseline
    drawLine(
        color = Grid,
        start = Offset(padLeft, chartH),
        end = Offset(size.width, chartH),
        strokeWidth = 1.5.dp.toPx()
    )
}

// legend
Row(
    Modifier.fillMaxWidth().padding(top = 4.dp),
    horizontalArrangement = Arrangement.Center
) {
    LegendDot(Crimson, "Credit")
    Spacer(Modifier.width(20.dp))
    LegendDot(Emerald500, "Payment")
}

}

//
=====
// Net position line chart
//
=====

@Composable
private fun NetLineChart(dailyData: List<DayData>) {
    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(180.dp)
    ) {
        val padLeft = 8.dp.toPx()
        val padBottom = 16.dp.toPx()
        val chartW = size.width - padLeft
        val chartH = size.height - padBottom
        val midY = chartH / 2f
    }
}

```

```

        val maxAbs = (dailyData.map { abs(it.net) }.maxOrNull() ?:
1.0).coerceAtLeast(1.0)
        val n      = dailyData.size

        // grid
        listOf(0.0f, 0.5f, 1.0f).forEach { frac ->
            drawLine(Grid, Offset(padLeft, chartH * frac),
Offset(size.width, chartH * frac), 1.dp.toPx())
        }

        // dashed zero line
        var dx = padLeft
        while (dx < size.width) {
            drawLine(
                color      = ZeroRef,
                start      = Offset(dx, midY),
                end        = Offset((dx +
8.dp.toPx()).coerceAtMost(size.width), midY),
                strokeWidth = 1.5.dp.toPx()
            )
            dx += 12.dp.toPx()
        }

        // line
        if (n > 1) {
            val xStep = chartW / (n - 1).toFloat()
            val path = Path()
            dailyData.forEachIndexed { i, d ->
                val x = padLeft + i * xStep
                val y = midY - (d.net / maxAbs).toFloat() * midY
                if (i == 0) path.moveTo(x, y) else path.lineTo(x, y)
            }
            drawPath(
                path = path,
                color = Blue,
                style = Stroke(width = 2.5.dp.toPx(), cap =
StrokeCap.Round, join = StrokeJoin.Round)
            )

            // dots
            dailyData.forEachIndexed { i, d ->
                val x = padLeft + i * xStep
                val y = midY - (d.net / maxAbs).toFloat() * midY
                drawCircle(Blue, radius = 4.dp.toPx(), center =
Offset(x, y))
                drawCircle(Color.White, radius = 2.dp.toPx(), center =
Offset(x, y))
            }
        }
    }
    Row(
        Modifier.fillMaxWidth().padding(top = 2.dp),
        horizontalArrangement = Arrangement.Center
    ) {
        LegendDot(Blue, "Net (Credit - Payment)")
    }
}

//

```

```

// Donut chart section

```

```
//
=====
@Composable
private fun DonutSection(outstanding: Double, collected: Double, rate:
Double) {
    val crimson = Crimson
    val emerald = Emerald500

    Row(
        modifier          = Modifier.fillMaxWidth(),
        verticalAlignment  = Alignment.CenterVertically
    ) {
        // donut
        Box(
            modifier          = Modifier.size(150.dp),
            contentAlignment = Alignment.Center
        ) {
            Canvas(Modifier.fillMaxSize()) {
                val stroke = 26.dp.toPx()
                val inset  = stroke / 2f + 4.dp.toPx()
                val arcBox = Size(size.width - inset * 2, size.height -
inset * 2)

                val tl      = Offset(inset, inset)
                val total   = outstanding + collected

                if (total <= 0.0) {
                    drawArc(Grid, 0f, 360f, false, tl, arcBox,
                        style = Stroke(stroke))
                } else {
                    val collAngle = (collected / total * 360f).toFloat()
                    val outAngle  = (outstanding / total * 360f).toFloat()
                    // collected arc (green) from top
                    if (collAngle > 0f) {
                        drawArc(emerald, -90f, collAngle, false, tl,
arcBox,
                            style = Stroke(stroke, cap = StrokeCap.Butt))
                    }
                    // outstanding arc (red)
                    if (outAngle > 0f) {
                        drawArc(crimson, -90f + collAngle, outAngle, false,
tl, arcBox,
                            style = Stroke(stroke, cap = StrokeCap.Butt))
                    }
                }
            }
            Column(horizontalAlignment = Alignment.CenterHorizontally) {
                Text("${"%0f".format(rate)}%",    fontWeight =
FontWeight.Bold, fontSize = 20.sp, color = Emerald500)
                Text("collected",    fontSize = 10.sp, color = Color.Gray)
            }
        }

        Spacer(Modifier.width(20.dp))

        // labels
        Column(verticalArrangement = Arrangement.spacedBy(14.dp)) {
            AmountLabel("● Outstanding",    "₹${"%0f".format(outstanding)}",
Crimson)
            AmountLabel("● Collected",    "₹${"%0f".format(collected)}",
Emerald500)
            HorizontalDivider()
        }
    }
}
```

```

        AmountLabel("Total Business", "₹${"%0f".format(outstanding +
collected)}", Color.Gray)
    }
}

@Composable
private fun AmountLabel(label: String, value: String, color: Color) {
    Column {
        Text(label, fontSize = 11.sp, color = color)
        Text(value, fontSize = 15.sp, fontWeight = FontWeight.Bold, color =
color)
    }
}

//
=====
// Top debtors (Compose layout bars – no Canvas needed)
//
=====
@Composable
private fun TopDebtorsChart(balances: List<CustomerBalance>) {
    val maxBal = (balances.maxOrNull { it.balance } ?:
1.0).coerceAtLeast(1.0)
    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
        balances.forEachIndexed { i, b ->
            val frac      = (b.balance / maxBal).toFloat().coerceIn(0.02f,
1f)

            val barAlpha  = 1f - i * 0.10f
            val barColor  = Crimson.copy(alpha = barAlpha)

            Row(
                modifier      = Modifier.fillMaxWidth(),
                verticalAlignment = Alignment.CenterVertically
            ) {
                // name
                Text(
                    text      = if (b.name.length > 11) b.name.take(11) +
"..." else b.name,
                    modifier = Modifier.width(84.dp),
                    fontSize  = 12.sp,
                    maxLines  = 1
                )
                Spacer(Modifier.width(6.dp))
                // track + fill
                Box(
                    modifier = Modifier
                        .weight(1f)
                        .height(20.dp)
                        .clip(RoundedCornerShape(4.dp))
                        .background(Color(0xFFFF1F5F9))
                ) {
                    Box(
                        modifier = Modifier
                            .fillMaxHeight()
                            .fillMaxWidth(frac)
                            .clip(RoundedCornerShape(4.dp))
                            .background(barColor)
                    )
                }
            }
        }
    }
}
=====

```



```

        Spacer(Modifier.width(8.dp))
        // amount
        Text(
            text = "₹${"%0f".format(b.balance)}",
            fontSize = 12.sp,
            fontWeight = FontWeight.SemiBold,
            color = Crimson,
            modifier = Modifier.width(64.dp),
            textAlign = TextAlign.End
        )
    }
}

//
//
// Summary stats card
//
//
@Composable
private fun SummaryCard(
    totalCredit : Double,
    totalPayment : Double,
    txnCount : Int,
    periodDays : Int,
    dailyData : List<DayData>
) {
    val activeDays = dailyData.count { it.credit > 0 || it.payment > 0 }
    val avgCredit = if (activeDays > 0) totalCredit / activeDays else 0.0
    val avgPayment = if (activeDays > 0) totalPayment / activeDays else 0.0
    val net = totalPayment - totalCredit

    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp), verticalArrangement =
Arrangement.spacedBy(0.dp)) {
            Text("Summary – Last $periodDays Days",
                fontWeight = FontWeight.SemiBold, fontSize = 15.sp)
            Spacer(Modifier.height(12.dp))

            listOf(
                Triple("Total Transactions", "$txnCount",
Color.Unspecified),
                Triple("Total Credit Given",
"₹${"%0f".format(totalCredit)}", Crimson),
                Triple("Total Payments Received",
"₹${"%0f".format(totalPayment)}", Emerald500),
                Triple("Net Change", "${if (net >= 0) "+" else
""}₹${"%0f".format(net)}", if (net >= 0) Emerald500 else Crimson),
                Triple("Avg Daily Credit",
"₹${"%0f".format(avgCredit)}", Color.Gray),
                Triple("Avg Daily Payment",
"₹${"%0f".format(avgPayment)}", Color.Gray),
                Triple("Active Days", "$activeDays /
$periodDays", Color.Gray)
            ).forEach { (label, value, color) ->
                Row(
                    modifier =
Modifier.fillMaxWidth().padding(vertical = 6.dp),
                    horizontalArrangement = Arrangement.SpaceBetween

```

```

        ) {
            Text(label, fontSize = 13.sp, color = Color.Gray)
            Text(
                value,
                fontSize = 13.sp,
                fontWeight = FontWeight.SemiBold,
                color = if (color == Color.Unspecified)
                    MaterialTheme.colorScheme.onSurface else color
            )
        }
        HorizontalDivider(color = Grid)
    }
}

//

// Shared helpers
//

@Composable
private fun LegendDot(color: Color, label: String) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Canvas(Modifier.size(10.dp)) { drawCircle(color) }
        Spacer(Modifier.width(5.dp))
        Text(label, fontSize = 11.sp, color = Color.Gray)
    }
}

```

Daily Log:

[Home](#) [Internships](#) [Companies](#) [About](#) [Contact](#) [Logout](#)

 **Student Dashboard**

 Dashboard

 Notifications

 Profile

 Applied Internships

 Saved Internships

 Internship Diary

 **Internship Diary Entries**

 Project

 Project Diary

 Project Diary Entries

 Project Team Members

 Custom Internships

 Change Password

Internship Diary Entries

Create

Date	Description	Actions
2026-05-03	On this day, I focused on validating the overall project design and ensuring its practicality in real-world scenarios. I revisited the requirement document and refined it to improve clarity, ensuring that each feature, user flow, and technical component was clearly defined and aligned with the project objectives. I analyzed how real users would interact with the application in daily use, particularly in environments with limited resources and low digital literacy, and made adjustments to ensure the application remains intuitive and easy to use. I also reviewed how different components such as customer management, transaction tracking, reporting, and potential advanced features fit together to form a cohesive system. Additionally, I evaluated edge cases and potential challenges, such as handling incomplete data, ensuring smooth offline functionality, and maintaining performance with increasing data. I ensured that the project roadmap is realistic and structured, with a clear sequence of implementation phases from core features to advanced functionalities. This step helped finalize the project plan and ensured that all aspects of the application are aligned and ready for development.	 
2026-05-02	On this day, I focused on aligning the technical architecture and overall implementation strategy of my project with the defined requirements and constraints. I reviewed the chosen architecture, ensuring that the MVVM pattern, Room database for offline-first functionality, and Jetpack Compose for UI development were appropriate and directly supported the project goals. I analyzed how data flows across layers from UI to ViewModel to repository and database and ensured that the structure remains clean, modular, and maintainable. In addition, I evaluated the user flow of the application and refined it to ensure simplicity and efficiency, especially considering the target users with low technical literacy. I also practiced the concept of daily alignment by reviewing tasks before implementation, ensuring that each activity contributes directly to core features rather than unnecessary enhancements. Furthermore, I identified potential areas of over-engineering and simplified design decisions to keep the application lightweight and focused. I categorized features into MVP and future enhancements, prioritizing essential functionalities such as transaction management, customer tracking, and ledger maintenance, while deferring advanced features like OCR and voice input for later stages. This step helped establish a balanced approach between functionality, performance, and development feasibility.	 
2026-05-01	Today, I worked on aligning the user flow of my application with the defined requirements. I carefully analyzed how a user would interact with the app, starting from opening the application to performing key actions such as adding transactions and viewing summaries. I ensured that each step in the user journey is logical, efficient, and directly supports the intended functionality. I simplified certain flows to reduce complexity and made sure that the process of completing tasks remains quick and intuitive, especially for users with limited technical experience.	 
2026-04-30	Today, I focused on identifying areas in my project where drift could potentially occur. I reviewed the list of planned features and critically evaluated whether each one directly contributes to solving the core problem. During this process, I identified certain features that were interesting but not essential for the initial version of the application. I removed or postponed these features and refined the requirement document to focus only on necessary functionalities. I ensured that each feature had a clear purpose and avoided including elements that were added merely for enhancement or visual appeal.	 

[Home](#) [Internships](#) [Companies](#) [About](#) [Contact](#) [Logout](#)

 **Student Dashboard**

 Dashboard

 Notifications

 Profile

 Applied Internships

 Saved Internships

 Internship Diary

 **Internship Diary Entries**

 Project

 Project Diary

 Project Diary Entries

 Project Team Members

 Custom Internships

 Change Password

2026-04-29	On this day, I attended a session conducted by Dr. Manish that emphasized the importance of maintaining alignment throughout the software development lifecycle. I began by carefully reviewing the problem statement of my project and understanding the core objectives it aims to achieve. I analyzed how each feature of the application contributes to solving the real-world issue and ensured that my initial understanding was not based on assumptions but grounded in the defined requirements. I also started mapping the high-level features to specific functional goals, ensuring that there is a direct connection between what I plan to build and the actual problem being addressed. This step helped me establish a strong conceptual foundation before moving into implementation.	 
2026-04-28	During this activity, I identified a real-world water wastage issue caused by continuous leakage that had already begun affecting nearby surfaces through dampness and deterioration. Instead of treating it as a minor inconvenience, I examined the situation more carefully to understand what was actually going wrong. The likely causes included hidden pipe damage, loose joints, or ineffective waterproofing, all of which allow water to escape even when the system is not actively in use. I documented the condition with clear before evidence, then ensured that corrective action was taken by addressing the leakage source and applying proper sealing or waterproofing measures. After the fix, I verified that the issue was resolved and documented the after condition to show the improvement. In parallel, I analyzed the working of the system itself how water flows through pipelines under pressure and how the same pressure, when combined with faults, leads to continuous unintended leakage. Finally, I consolidated all observations, analysis, actions, and results into a well-structured report and submitted it, making sure it clearly reflected both the problem-solving process and the outcome.	 
2026-04-27	On this day, I focused on understanding the problem statement and defining the complete project requirements for the Namma-Santhe Ledger application. I analyzed the real-world challenges faced by small vendors in village markets, particularly in tracking credit (udari) and managing repayments using traditional methods. Based on this, I structured a detailed requirement document covering core features such as customer management, transaction tracking, and ledger maintenance, along with advanced functionalities like OCR-based bill scanning, voice input, and PDF invoice generation. I also defined the system architecture using MVVM and planned the use of technologies such as Room Database for offline-first functionality, Jetpack Compose for UI, and WorkManager for background tasks. Additionally, I outlined the user flow, database design, screen structure, and performance constraints to ensure the app remains simple, fast, and suitable for low-literacy users. This step helped in establishing a clear roadmap and prioritizing features from MVP to advanced implementation.	 
2026-04-26	On these days, I focused on applying AI agents in real-world business scenarios and understanding their deployment using Salesforce Agentforce. I worked on building and customizing a Sales Coach agent that provides real-time guidance to improve sales communication, refine pitching strategies, and enhance customer interaction. The agent was designed to analyze user inputs and generate context-aware suggestions, helping simulate real-world sales assistance. In addition, I implemented an automated quote generation system where the agent produces structured business quotes based on predefined rules and inputs, ensuring consistency and alignment with business logic. I integrated prompt templates with workflows and agent actions, enabling the system to not only generate responses but also execute meaningful tasks through backend automation. Furthermore, I explored how AI agents support customer engagement at scale by handling repetitive tasks, assisting users with recommendations, and improving operational efficiency. I also studied the deployment process of AI agents, including testing workflows, validating outputs, and ensuring proper integration of components such as prompts, flows, and actions. Additionally, I analyzed how agents are scaled for enterprise use and explored multi-channel capabilities such as voice-based interactions, understanding how AI systems can operate across different platforms to provide a seamless user experience.	 


[Home](#)
[Internships](#)
[Companies](#)
[About](#)
[Contact](#)

Logout

to provide a seamless user experience.

13 Student Dashboard

Dashboard

Notifications

Profile

Applied Internships

Saved Internships

Internship Diary

Internship Diary Entries

Project

Project Diary

Project Diary Entries

Project Team Members

Custom Internships

Change Password

2026-04-25

On this day, I focused on integrating Agent Actions with service agents and customizing their behavior using prompts, flows, and topics. I configured agents to trigger backend workflows using Flow automation, allowing them to perform actions such as retrieving data, updating records, or executing predefined business logic. I also worked on customizing a service agent by defining conversation topics, linking them with flows, and attaching prompt templates to guide responses. Through the Agentforce Service Assistant module, I analyzed how AI agents handle customer queries, manage interactions, and provide contextual responses. This involved understanding how multiple components prompts, flows, and actions work together to create a fully functional conversational system.




2026-04-24

On this day, I worked extensively with Prompt Builder in Salesforce Agentforce to design structured and reusable prompt templates for enterprise use cases. I created a sales email prompt template that dynamically generates personalized emails based on input variables such as customer name, product details, and intent. I also explored the Superbadge: Prompt Builder Templates, where I designed flexible templates capable of adapting to different business contexts. This involved defining input parameters, controlling tone and format of responses, and ensuring consistency across generated outputs. Additionally, I analyzed how prompt templates can be integrated into workflows so that agents can use them in real-time scenarios, improving automation and response efficiency.




2026-04-23

On this day, I focused on the sales-oriented capabilities of Agentforce, including Sales Coach and quote generation systems. I explored how AI agents can assist sales teams by providing real-time coaching, improving communication strategies, and enhancing customer interactions. I also worked on building a sales coaching agent that helps users refine their pitch and approach. Additionally, I implemented a system for generating quotes automatically based on predefined business rules, demonstrating how AI can streamline sales processes. This session highlighted how AI can be applied in business environments to improve efficiency and decision-making.




2026-04-22

On this day, I explored advanced features of Agentforce including Agentforce Voice and deployment strategies. I learned how AI agents can be extended beyond text-based interfaces to voice-based interactions, enabling more natural and accessible communication. I studied how to configure voice-enabled agents and understand their use cases in customer support and business operations. Additionally, I explored deployment processes, learning how AI agents are rolled out to users in production environments. This included understanding testing, validation, and monitoring of agent performance.




2026-04-21

On this day, I worked on customizing service agents using prompts, flows, and actions in Salesforce Agentforce. I explored how to enhance agent capabilities by defining topics, configuring flows, and designing prompt templates that guide agent responses. I also studied how service agents can handle real-world customer interactions such as inquiries, issue resolution, and bookings. Additionally, I worked on the Superbadge: Agentforce Service, which involved building a fully functional service agent capable of managing customer requests efficiently. This task required integrating multiple components such as prompts, automation flows, and action handling.




2026-04-20

On this day, I focused on understanding and implementing Agent Actions within the Salesforce Agentforce platform. I explored how AI agents can trigger automated workflows using Flow integration, enabling them to perform real-time actions based on user queries. I worked on configuring actions that allow agents to interact with backend systems and execute predefined business logic. Additionally, I learned how conversational AI can be enhanced by connecting it with enterprise workflows, making agents more functional and responsive. This helped me understand how AI is not just limited to generating responses but can actively perform tasks within an organization.




Certificate: